

http://algs4.cs.princeton.edu

5.3 SUBSTRING SEARCH

- *introduction*
- *brute force*
- *Knuth-Morris-Pratt*
- *Boyer-Moore*
- *Rabin-Karp*



5.3 Substring Search

- *introduction*
- *brute force*
- *Knuth-Morris-Pratt*
- *Boyer-Moore*
- *Rabin-Karp*

Substring search

Qëllimi. Të gjindet pattern me length M në një tekst me length N.

typically N >> M

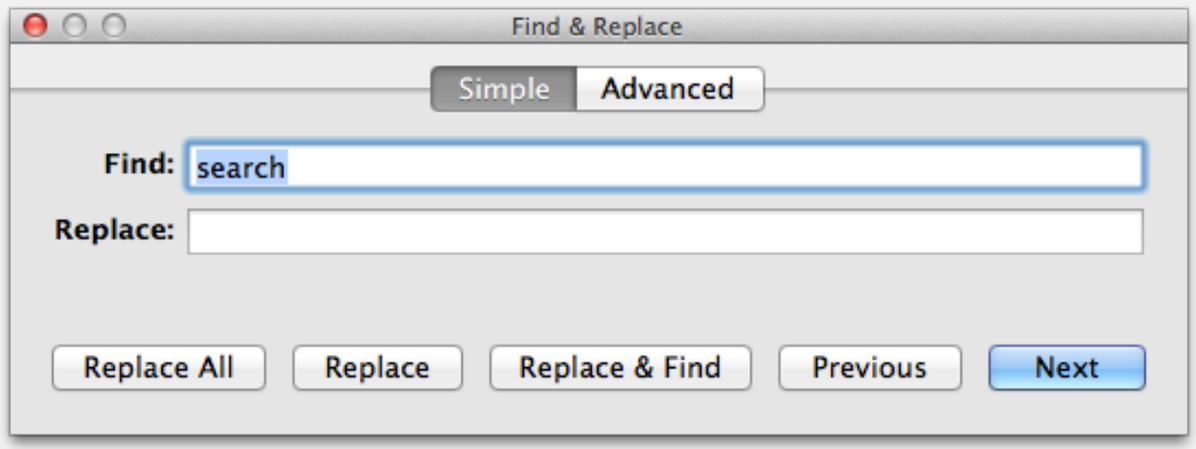


Substring search - aplikimi

Qëllimi. Të gjindet pattern me length M në një tekst me length N.

typically N >> M





Substring search - aplikimi

Qëllimi. Të gjindet pattern me length M në një tekst me length N.

typically N >> M



Forensika kompjuterike. Kërkohet memoria ose disku për URL ose RSA tastet që përdoruesi ka shtypur.



http://citp.princeton.edu/memory

Substring search - aplikimi

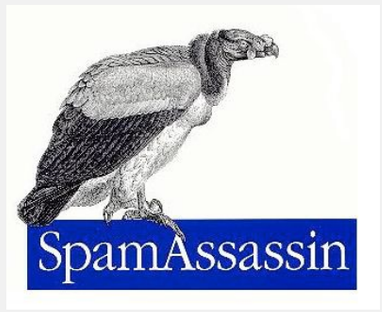
Qëllimi. Të gjindet pattern me length M në një tekst me length N.

typically N >> M



Identifikohen patterns që indikojnë spam.

- PROFITS
- LOSE WE1GHT
- There is no catch.
- This is a one-time mailing.
- This message is sent in compliance with spam regulations.





Substring search - aplikimi

Përgjimi elektronik.





Duhet të monitorohet i gjithë trafiku i. (siguria)

Nuk ka gjasa! (privatësia)



Ne jemi kryesisht të interesuar për “ATTACK AT DAWN”

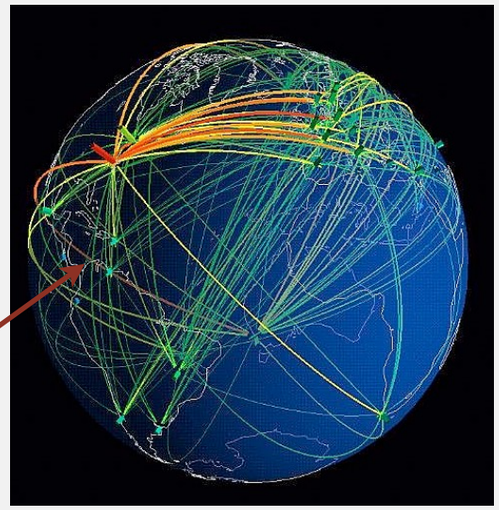


OK. Ndërtoni një maqine që kërkon VETËM për atë.



“ATTACK AT DAWN” substring search maqina

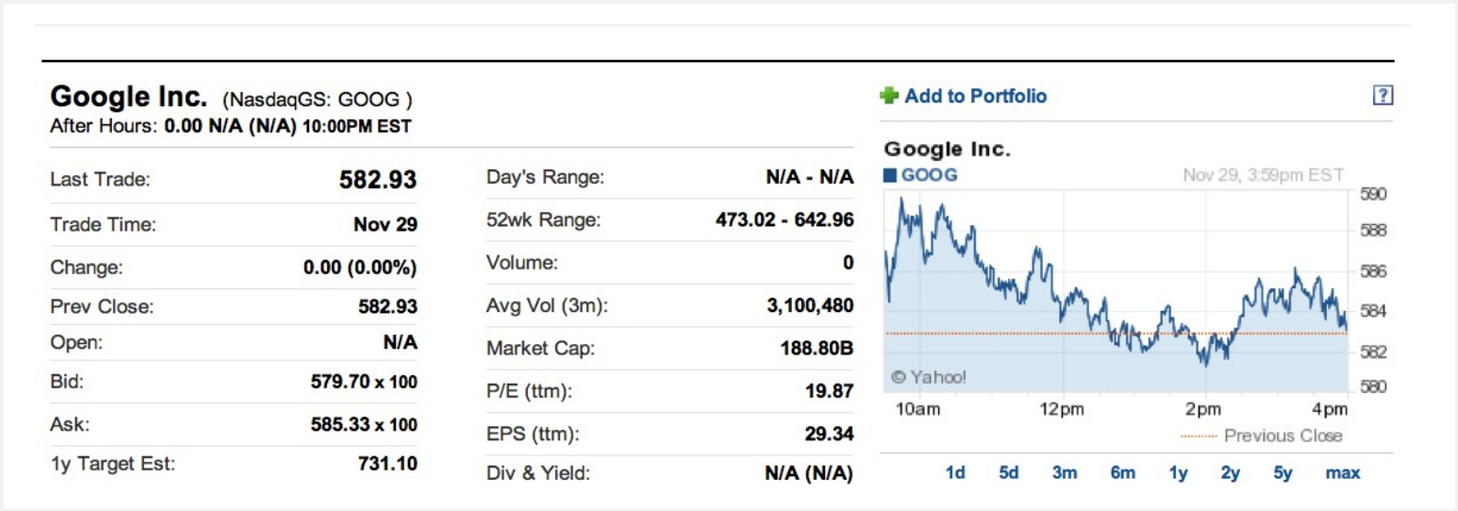
Është gjetë



Substring search - aplikimi

Screen scraping. Nxirren të dhëna relevante nga web faqja.

P.Sh. Të gjindet string-u I kufizuar me `<b>` dhe `</b>` pas paraqitjes së pare të pattern `Last Trade:`.



http://finance.yahoo.com/q?s=goog

```
...
<tr>
<td class= "yfnc_tablehead1"
width= "48%">
Last Trade:
</td>
<td class= "yfnc_tabledata1">
<big><b>452.92</b></big>
</td></tr>
<td class= "yfnc_tablehead1"
width= "48%">
Trade Time:
</td>
<td class= "yfnc_tabledata1">
...
```

Screen scraping: Java implementimi

Java library. Metoda `indexOf()` në Java's string library kthen index of të paraqitjes së pare të një string-u të dhënë, duke filluar nga një offset.

```
public class StockQuote
{
   public static void main(String[] args)
   {
      String name = "http://finance.yahoo.com/q?s=";
      In in = new In(name + args[0]);
      String text = in.readAll();
      int start    = text.indexOf("Last Trade:", 0);
      int from     = text.indexOf("<b>",  start);
      int to       = text.indexOf("</b>", from);
      String price = text.substring(from + 3, to);
      StdOut.println(price);
   }
}
```

```
% java StockQuote goog
582.93

% java StockQuote msft
24.84
```

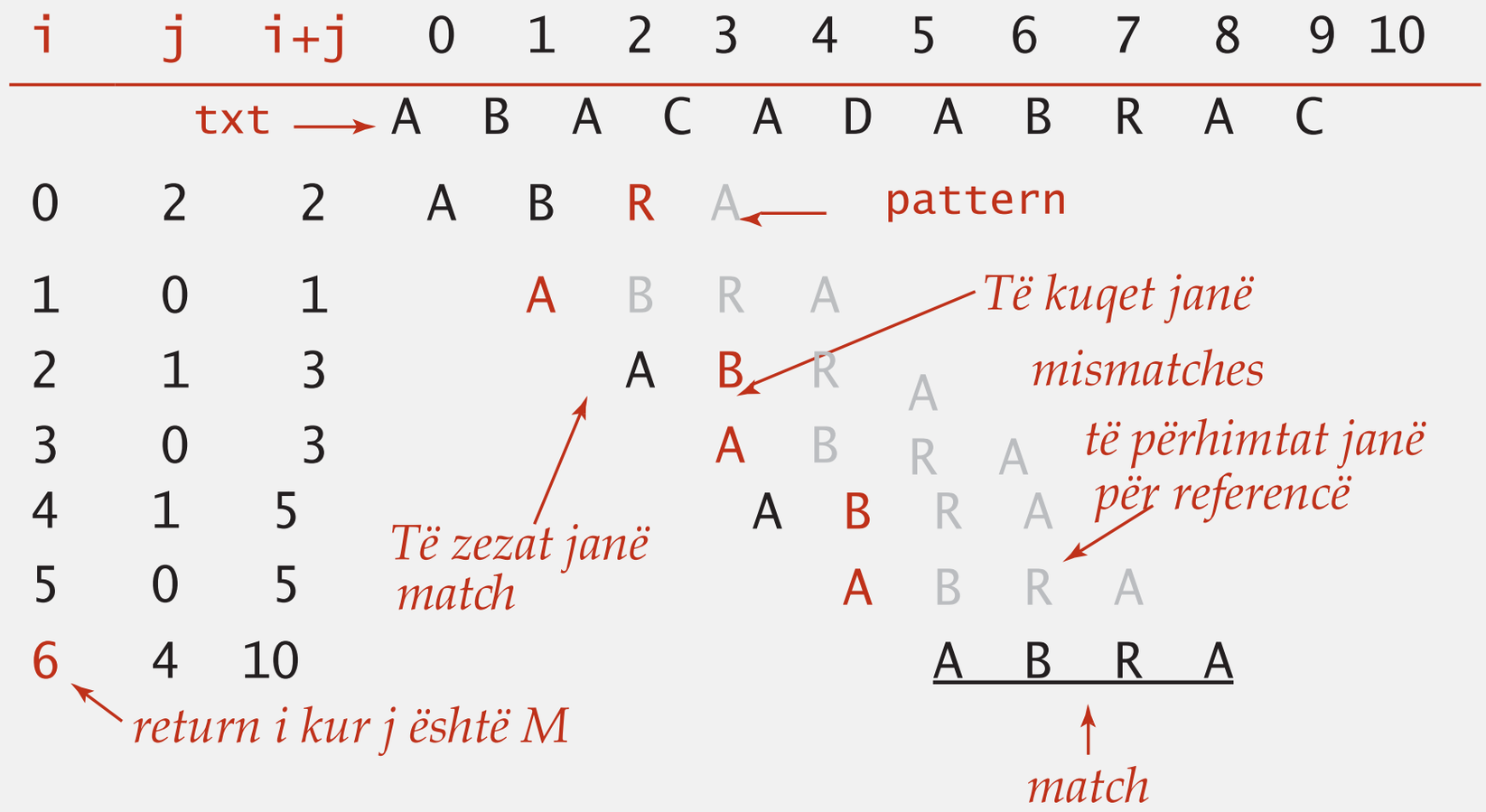


5.3 Substring Search

- *introduction*
- *brute force*
- *Knuth-Morris-Pratt*
- *Boyer-Moore*
- *Rabin-Karp*

Brute-force substring search

Kontrollohet për pattern duke filluar në çdo pozitë në tekst.



Brute-force substring search:Java implementimi

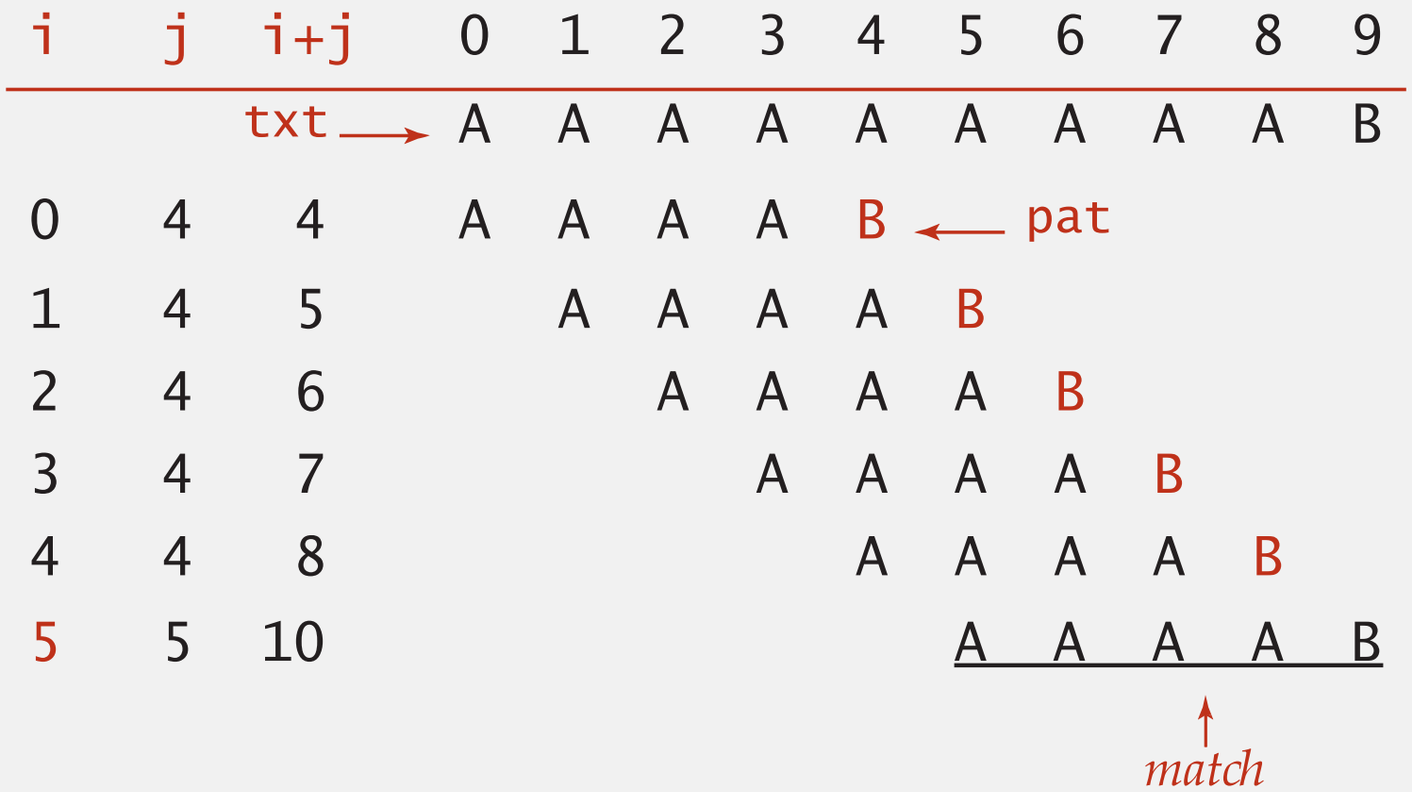
Kontrollohet për pattern duke filluar në çdo pozitë në tekst.

i	j	i + j	0	1	2	3	4	5	6	7	8	9	10
			A	B	A	C	A	D	A	B	R	A	C
4	3	7					A	D	A	C	R		
5	0	5						A	D	A	C	R	

```
public static int search(String pat, String  txt)
{
   int M = pat.length();
   int N = txt.length();
   for (int i = 0; i <= N - M; i++)
   {
      int j;
      for (j = 0; j < M; j++)
         if (txt.charAt(i+j) != pat.charAt(j))
            break;
      if (j == M) return i;   <-- index in text where pattern starts
   }
   return N;                  <-- not found
}
```

Brute-force substring search: Rasti më i keq

Brute-force algoritmi mund të jetë i ngadalshëm kur teksti dhe pattern përsëriten.



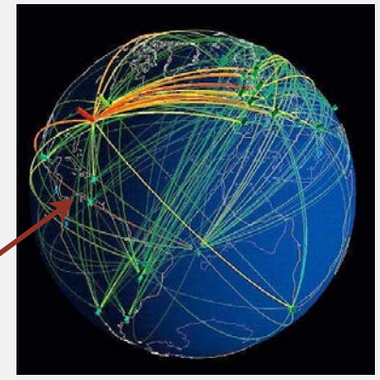
Rasti më i keq ~ kur ka $M\ N$ char krahasime.

.

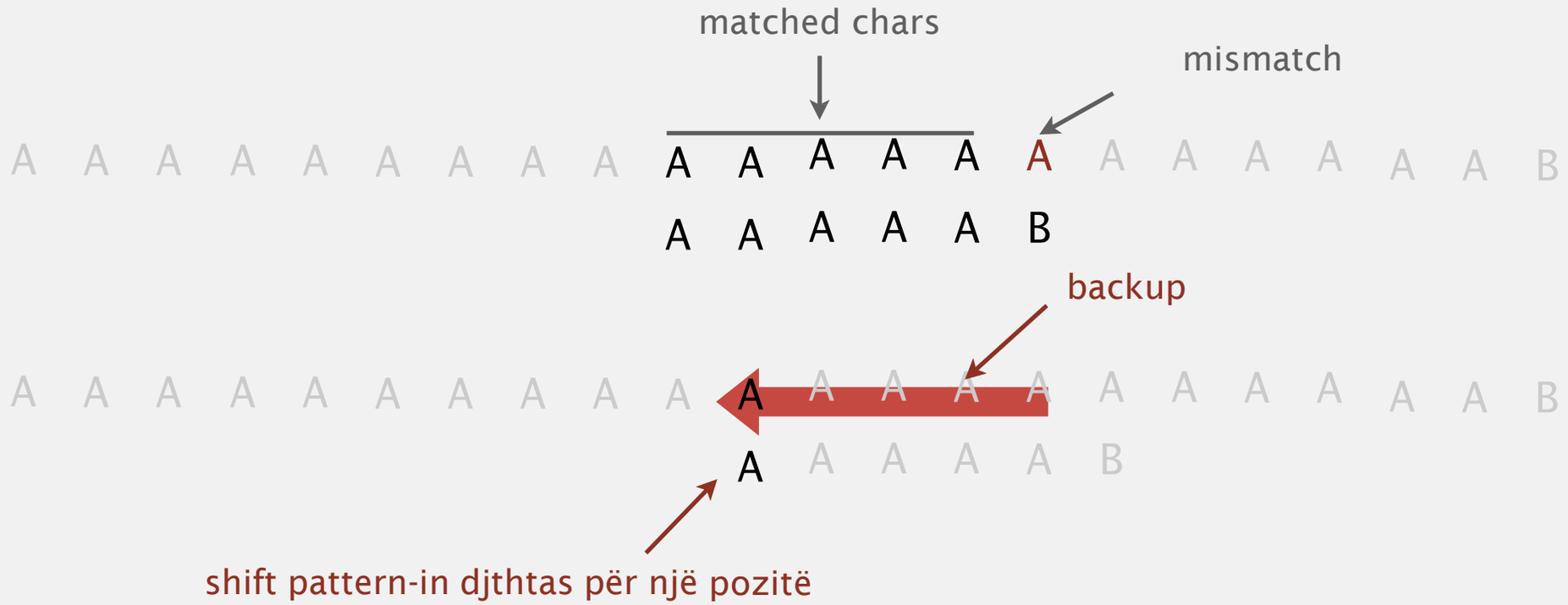
Backup

Në shumë aplikacione, tentohet të ikët nga backup në tekst.

- Input-i trajtohet si stream i të dhënave.
- Modeli abstrakt: standard input.



Brute-force algoritm-it i duhet backup për çdo mismatch.



Qasja 1. Të mbahet një buffer i M char të fundit.

Qasja 2. Në vijim.

Brute-force substring search : implementim alternativ

Sekuenca e njejtë e char krahasohet si në implementimin e kaluar.

- `i` tregon fundin e sekuencës e char që veç janë match në tekst.
- `j` ruan # e match char (fundi i sekuencës në pattern).

i	j	0	1	2	3	4	5	6	7	8	9	10
		A	B	A	C	A	D	A	B	R	A	C
7	3					A	D	A	C	R		
5	0						A	D	A	C	R	

```
public static int search(String pat, String txt)
{
   int i, N =
   txt.length(); int j, M
   = pat.length();
   for (i = 0, j = 0; i < N && j < M; i++)
   {
      if (txt.charAt(i) == pat.charAt(j)) j++;
      else { i -= j; j = 0;  }

}  }
   if (j == M) return i -
   M; else          return N;
```

Backup eksplicit

Mangësitë algoritmike në substring search

Brute-force nuk është gjithmonë e përshtatshme.

Mangësitë teorike. Linear-time garantohet. ← Problem fundamental në algoritme

Mangësitë praktike. Backup në tekst stream. ← Shpesh nuk ka vend ose kohë për të ruajtur tekstin

```
Now is the time for all people to come to the aid of their party. Now is the time for all good people to
come to the aid of their party. Now is the time for many good people to come to the aid of their party.
Now is the time for all good people to come to the aid of their party. Now is the time for a lot of good
people to come to the aid of their party. Now is the time for all of the good people to come to the aid of
their party. Now is the time for all good people to come to the aid of their party. Now is the time for
each good person to come to the aid of their party. Now is the time for all good people to come to the aid
of their party. Now is the time for all good Republicans to come to the aid of their party. Now is the
time for all good people to come to the aid of their party. Now is the time for many or all good people to
come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now
is the time for all good Democrats to come to the aid of their party. Now is the time for all people to
come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now
is the time for many good people to come to the aid of their party. Now is the time for all good people to
come to the aid of their party. Now is the time for a lot of good people to come to the aid of their
party. Now is the time for all of the good people to come to the aid of their party. Now is the time for
all good people to come to the aid of their attack at dawn party. Now is the time for each person to come
to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is
the time for all good Republicans to come to the aid of their party. Now is the time for all good people
to come to the aid of their party. Now is the time for many or all good people to come to the aid of their
party. Now is the time for all good people to come to the aid of their party. Now is the time for all good
Democrats to come to the aid of their party.
```



5.3 Substring Search

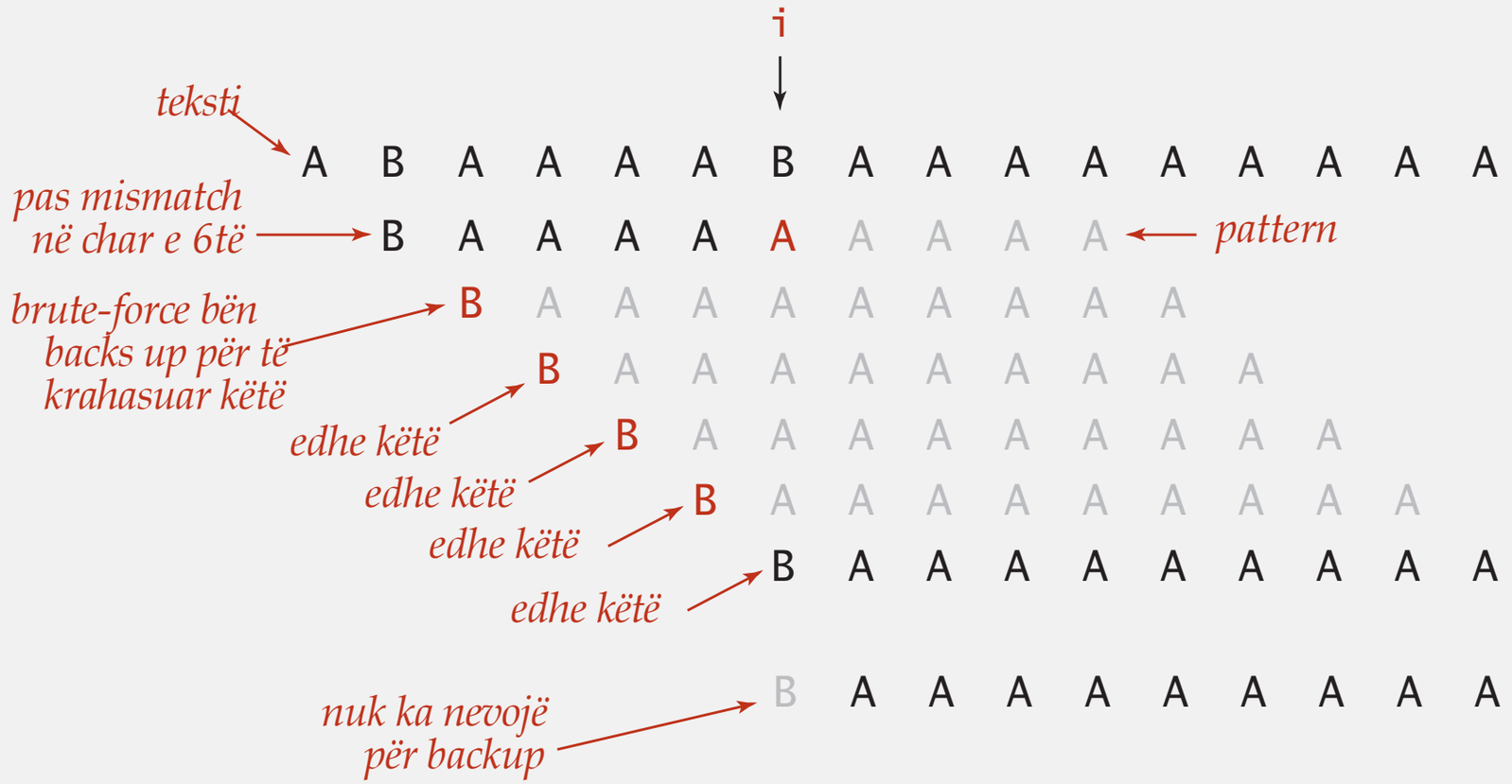
- *introduction*
- *brute force*
- *Knuth-Morris-Pratt*
- *Boyer-Moore*
- *Rabin-Karp*

Knuth-Morris-Pratt substring search

Intuition. Le të tentohet të gjindet (search) në tekst për pattern BAAAAAAAAA.

- Supozohet që ka match për 5 char në pattern, kurse mismatch në char e 6të
- Dihet që 6 char e kaluara në tekst janë BAAAAB.
- S'ka nevojë për back up të tekst pointer-it!

supozohet { A, B } alfabet



Knuth-Morris-Pratt algoritmi. Mënyrë e mire për t'iu shmangur gjithmonë backup-it. (!)

Deterministic finite state automaton (DFA)

DFA është string-searching maqinë abstrakte.

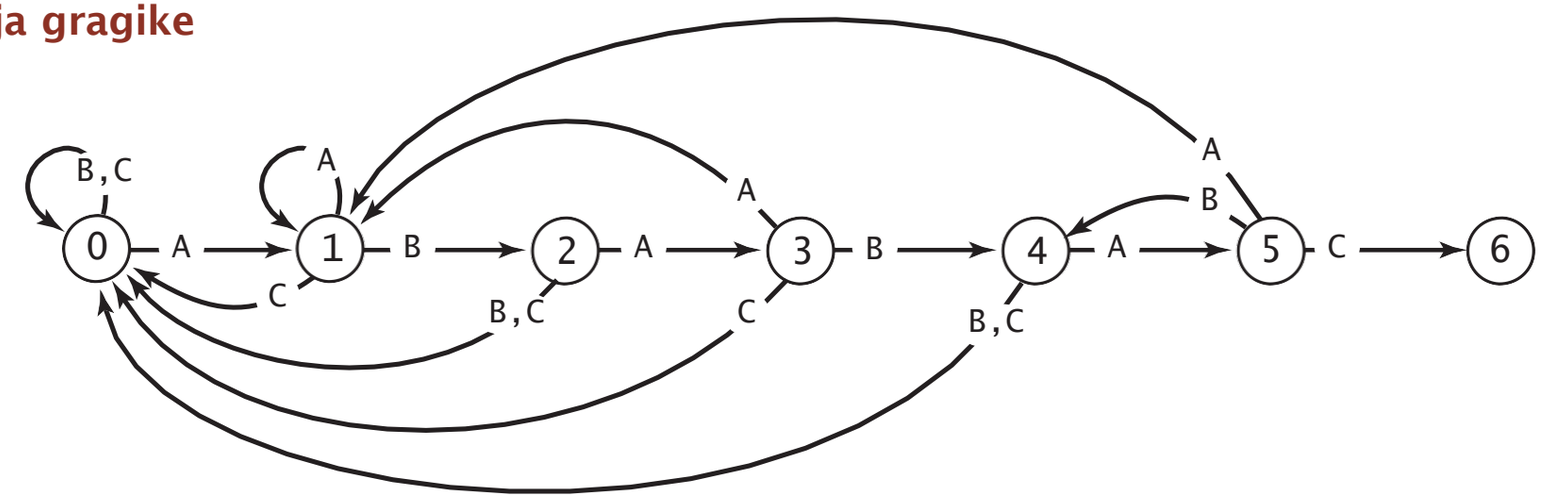
- Numër i fundëm (finite) i gjendjeve (përfshirë start dhe fund (halt)).
- Saktësisht një transicion për çdo char në alfabet.
- Accept nëse sekuenca e transivioneve çon deri në gjendjen e fundit (halt).

paraqitja interne

	j	0	1	2	3	4	5
pat.charAt(j)		A	B	A	B	A	C
dfa[][j]	A	1	1	3	1	5	1
	B	0	2	0	4	0	4
	C	0	0	0	0	0	6

if në gjendjen `j` lexohet char `c`:
  if `j` është 6 halt dhe accept
else kalo në gjendjen `dfa[c][j]`

paraqitja gragike



Knuth-Morris-Pratt demo: DFA simulimi

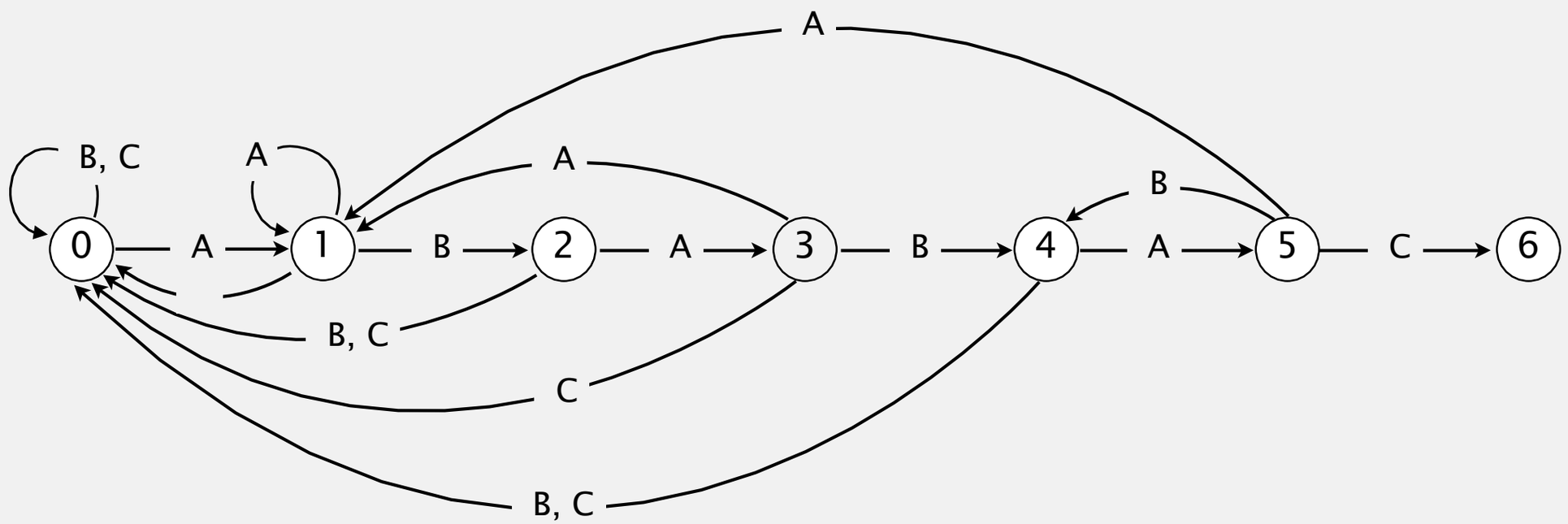
A A B A C A A B A B A C A A



pat.charAt(j)

dfa[][j]

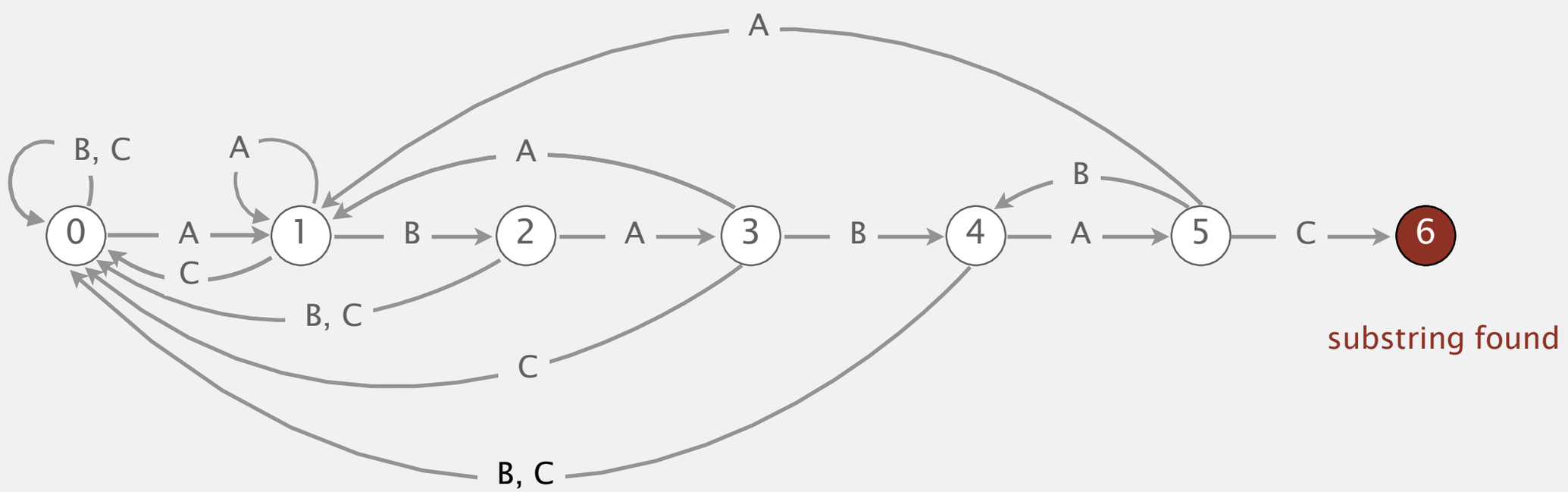
	0	1	2	3	4	5
	A	B	A	B	A	C
A	1	1	3	1	5	1
B	0	2	0	4	0	4
C	0	0	0	0	0	6



Knuth-Morris-Pratt demo: DFA simulimi

A A B A C A A B A B A C A A

pat.charAt(j)	0	1	2	3	4	5
	A	B	A	B	A	C
dfa[][j] A	1	1	3	1	5	1
dfa[][j] B	0	2	0	4	0	4
dfa[][j] C	0	0	0	0	0	6



substring found

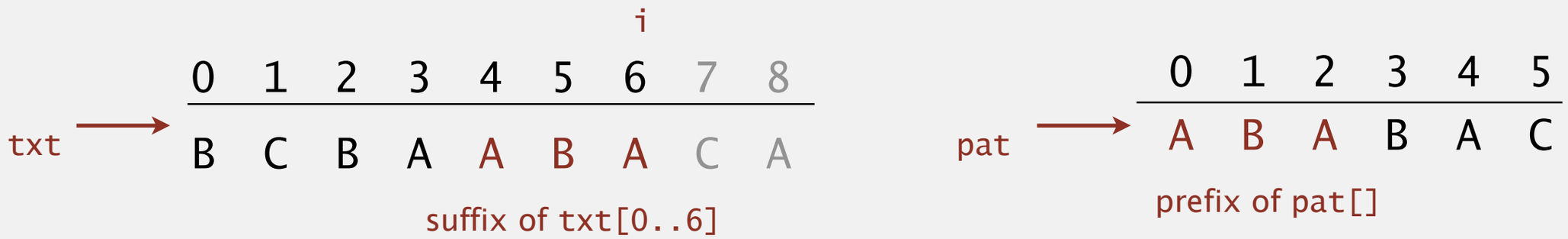
Interpretimi i Knuth-Morris-Pratt DFA

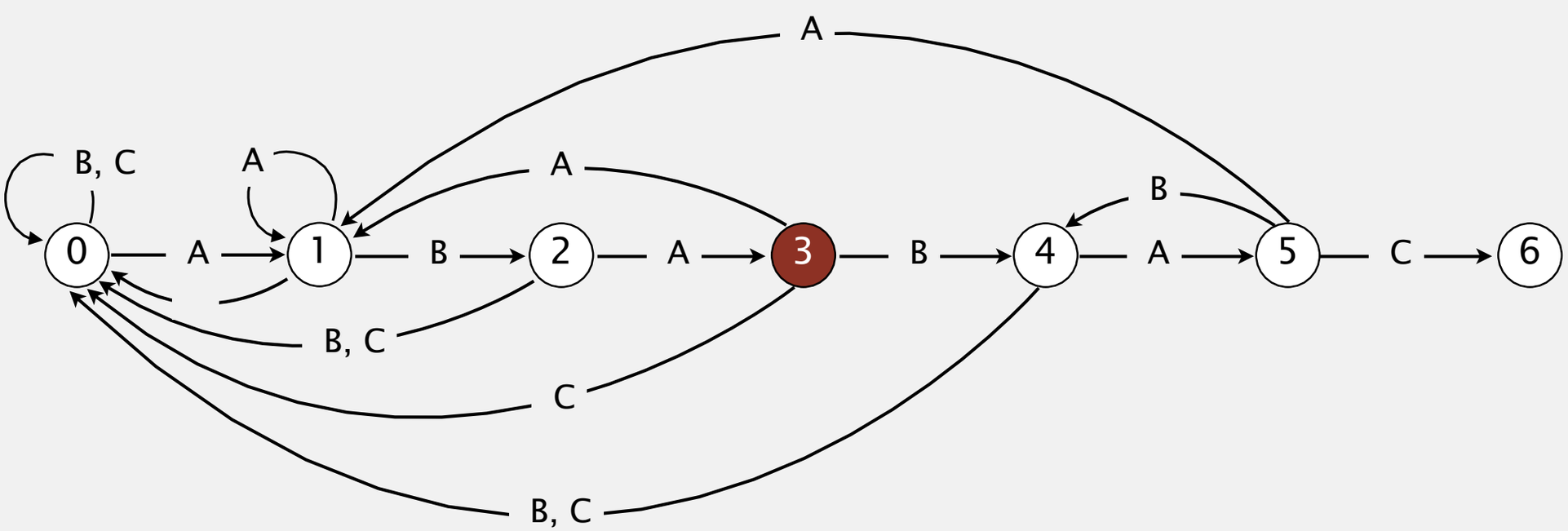
Pyetje. Cili është interpretimi i DFA gjendjes pas leximit në `txt[i]`?

Përgjigje. Gjendja = numri i karaktereve në pattern që janë match.

length i prefiksit më të gjatë të `pat[]` që i bie sufiksi i `txt[0..i]`

P.sh. DFA është në gjendjen 3 pas leximit në `txt[0..6]`.





Knuth-Morris-Pratt substring search: Java implementimi

Dallimet kryesore në implementim krahasuar me brute-force.

- Duhet të parallogaritet dfa[][] nga pattern.
- Tekst pointer-i kurrë nuk zvogëlohet (decrement).

```
public int search(String txt)  j < M; i++)
{
   int i, j, N = txt.length();
   for (i = 0, j = 0; i < N &&
      j = dfa[txt.charAt(i)][j];
   if (j == M) return i - M;
   else        return N;
}
```

pa backup

Kohëzgjatja.

- Simulate DFA on text: at most N character accesses.
- Build DFA: how to do efficiently?

[warning: tricky algorithm ahead]

Knuth-Morris-Pratt substring search: Java implementimi

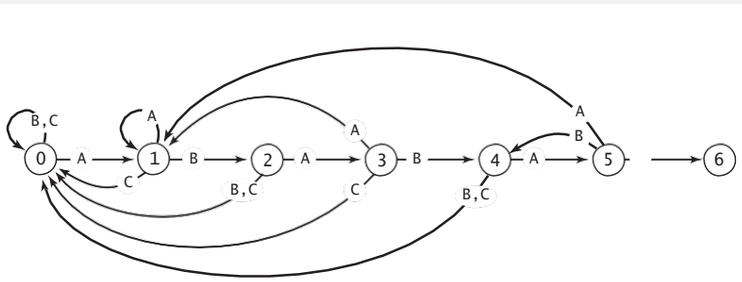
Dallimet kryesore në implementim krahasuar me brute-force.

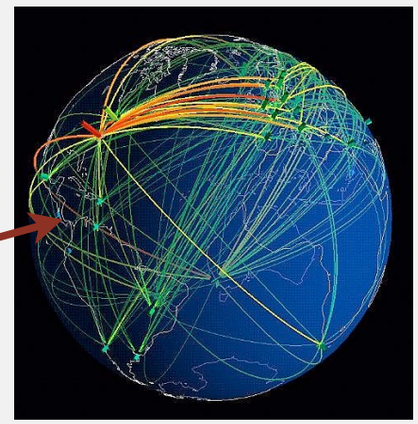
- Duhet të parallogaritet dfa[][] nga pattern.
- Tekst pointer-i kurrë nuk zvogëlohet (decrement).
- Mund të përdoret input stream-i.

```
public int search(In in)
{
   int i, j;
   for (i = 0, j = 0; !in.isEmpty() && j < M; i++)
      j = dfa[in.readChar()][j];

   if (j == M) return i - M;
   else        return NOT_FOUND;
}
```

pa backup





Knuth-Morris-Pratt demo: DFA - konstruktimi

Shtohet një gjendje për secilin karakter në pattern (plus accept gjendja).



1	2	3	4	5
B	A	B	A	C

pat.charAt(j)
A
B
C

dfa[][j]

Konstruktimi i DFA për KMP substring search për A B A B A C pattern

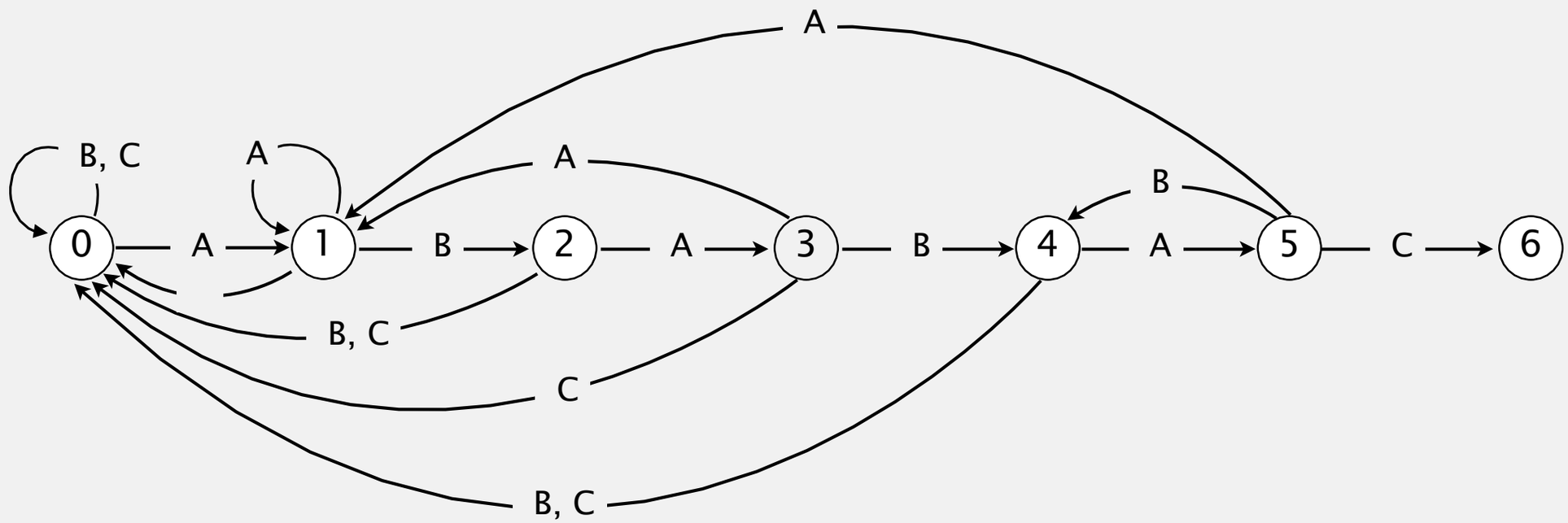


Knuth-Morris-Pratt demo: DFA - konstruktimi

pat.charAt(j)	0	1	2	3	4	5
	A	B	A	B	A	C
A	1	1	3	1	5	1
B	0	2	0	4	0	4
C	0	0	0	0	0	6

dfa[][j]

Konstruktimi i DFA për KMP substring search për A B A B A C pattern



Ndërtimi i DFA nga një pattern

Shtohet një gjendje për secilin karakter në pattern (plus accept gjendja).

	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C

dfa[][j]

A	
B	
C	



Ndërtimi i DFA nga një pattern

Match transicioni. If në gjendje `j` dhe tjetri char `c` == `pat.charAt(j)`, shko tek `j+1`.

first j characters of pattern have already been matched

next char matches

now first j +1 characters of pattern have been matched

`pat.charAt(j)`

	0	1	2	3	4	5
	A	B	A	B	A	C
A	1		3		5	
B		2		4		
C						6

`dfa[][j]`



Knuth-Morris-Pratt demo: konstruktimi i DFA në kohë lineare

Shtohet një gjendje për secilin karakter në pattern (plus accept gjendja).



	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
dfa[][j]	A					
	B					
	C					

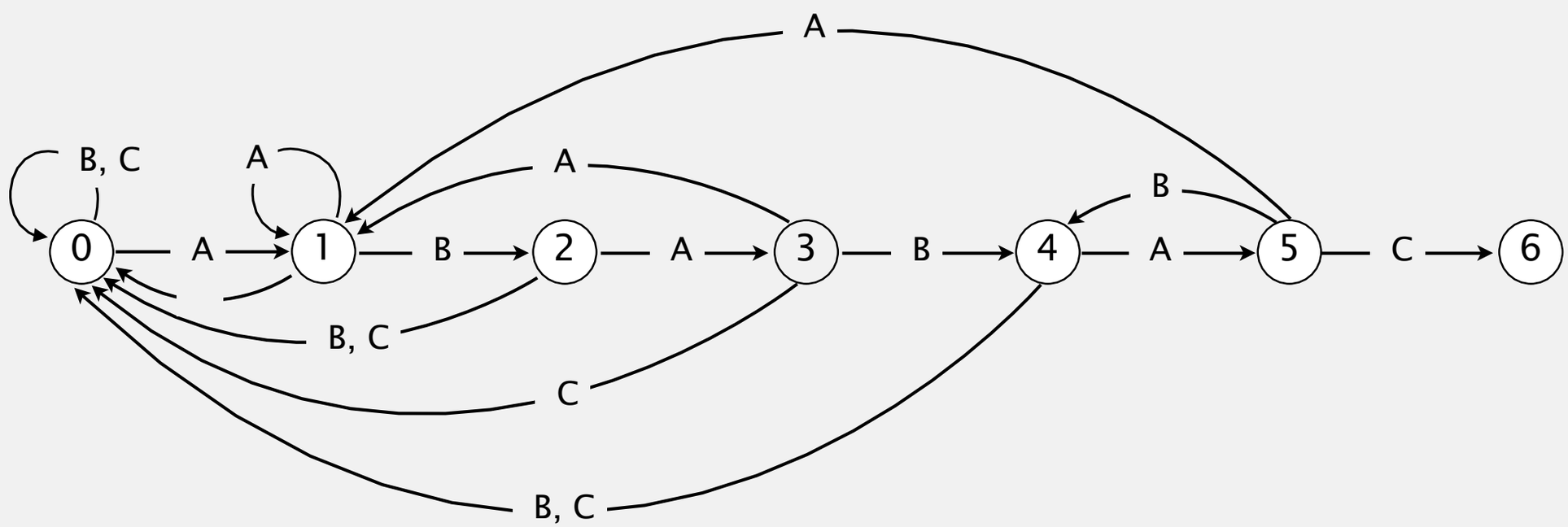
Konstruktimi i DFA për KMP substring search për A B A B A C pattern



Knuth-Morris-Pratt demo: konstruktimi i DFA në kohë lineare

dfa[][j]	(j)	0	1	2	3	4	5
pat.charAt		A	B	A	B	A	C
	A	1	1	3	1	5	1
	B	0	2	0	4	0	4
	C	0	0	0	0	0	6

Konstruktimi i DFA për KMP substring search për A B A B A C pattern





http://algs4.cs.princeton.edu

5.3 Substring Search

- *introduction*
- *brute force*
- *Knuth-Morris-Pratt*
- *Boyer-Moore*
- *Rabin-Karp*



Robert Boyer



J. Strother Moore

Boyer-Moore

Intuition.

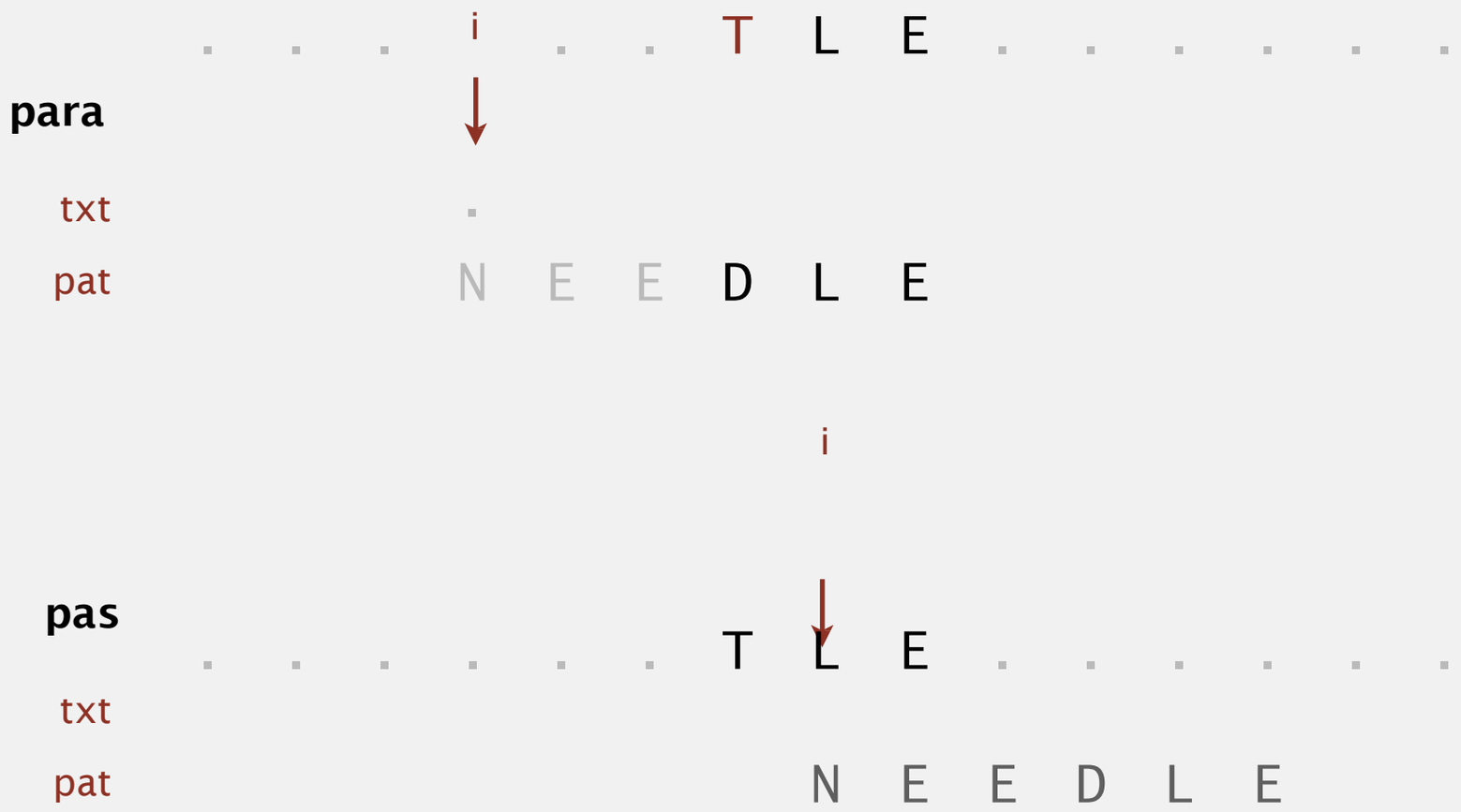
- Skenohën karakteret në pattern nga e djathta në të majtë.
- Muns të kalohen deri në M tekst karaktere kur të gjindet një që nuk I përket pattern.



Boyer-Moore

Pyetje. Sa karaktere të kalohen?

Rasti 1. Mismatch char nuk gjindet në pattern.

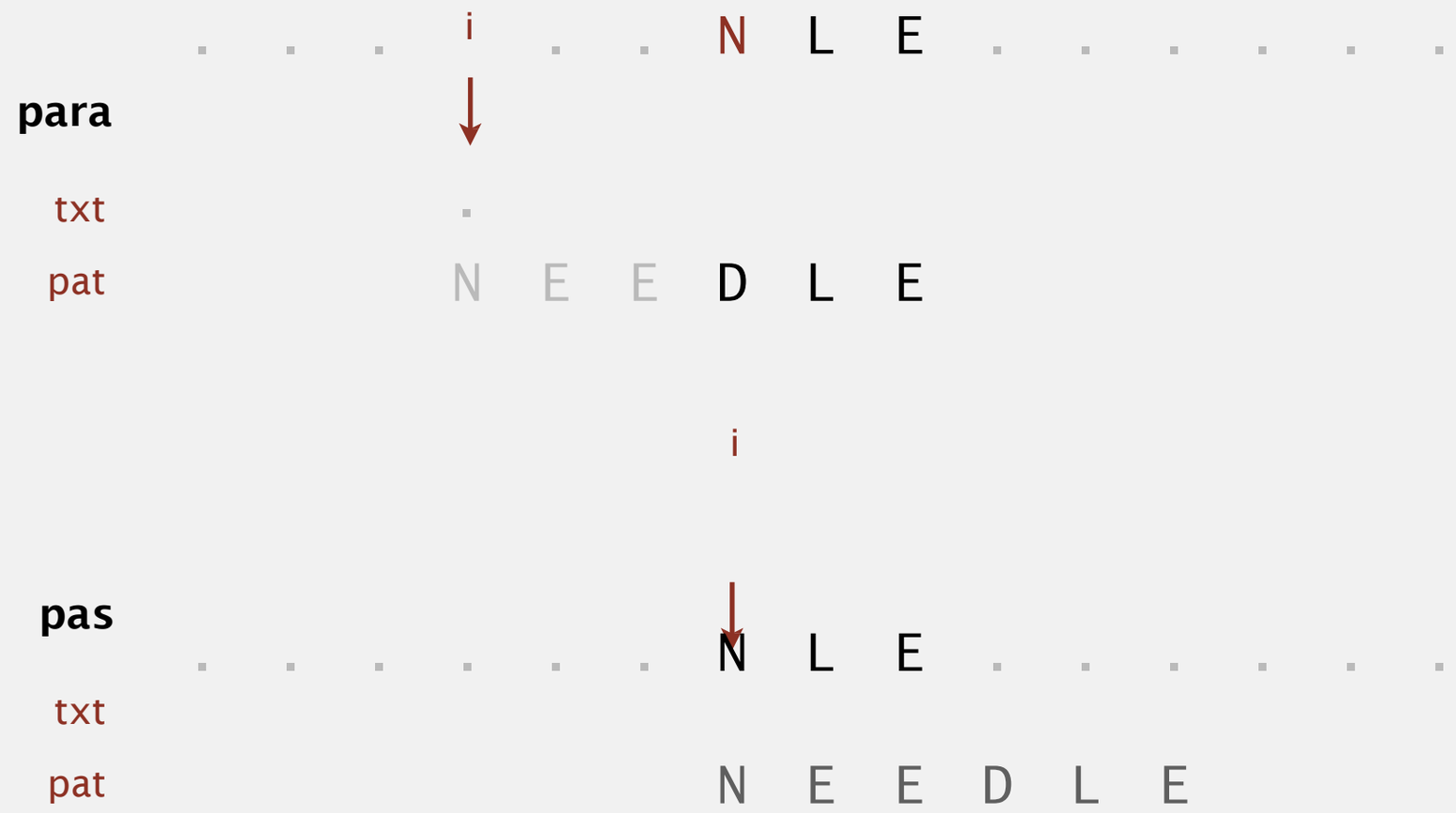


mismatch char 'T' nuk gjindet në pattern: increment i një char pas 'T'

Boyer-Moore

Pyetje. Sa karaktere të kalohen?

Rasti 2a. Mismatch char nuk gjindet në pattern.

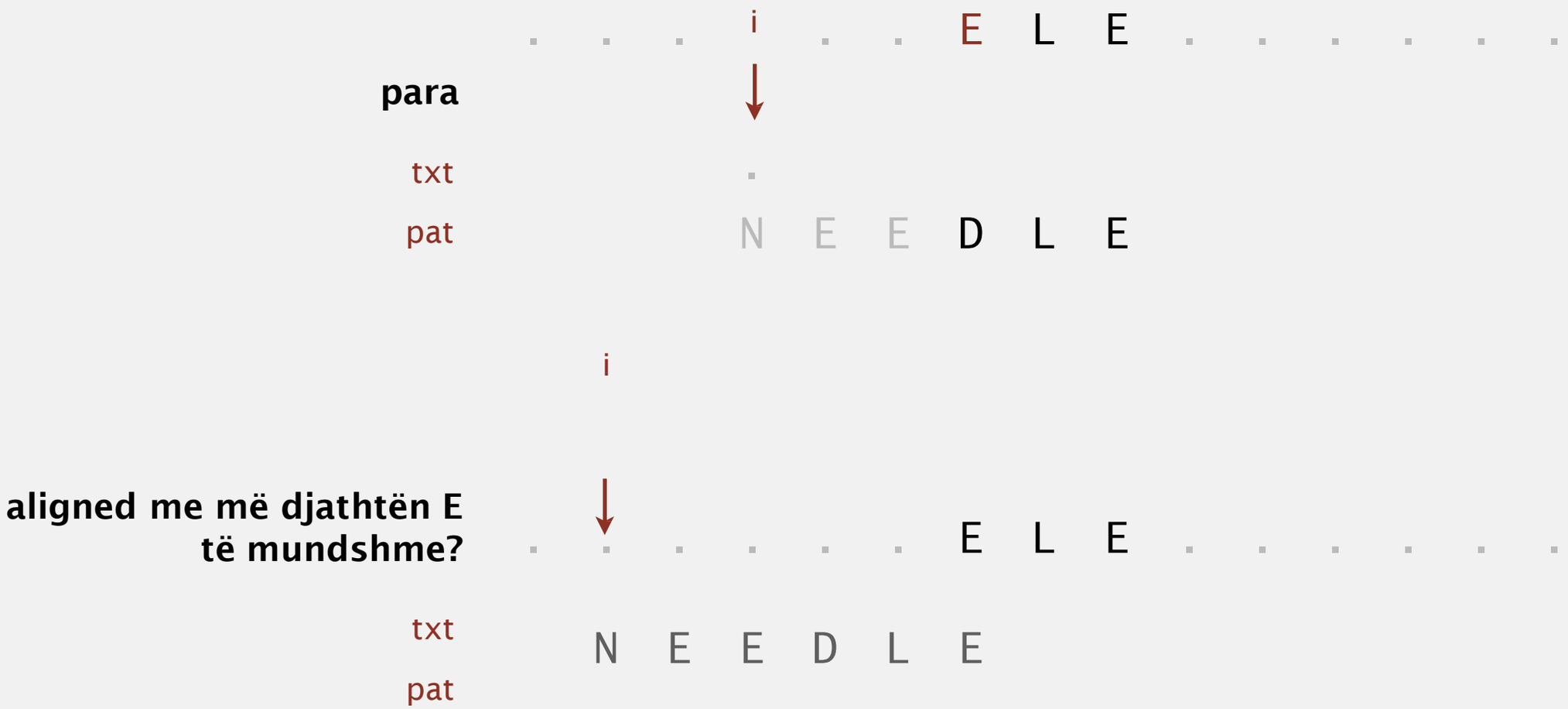


mismatch char 'N' nuk gjindet në pattern: **align 'N' të tesktit me 'N' më të djathtë në pattern**

Boyer-Moore

Pyetje. Sa karaktere të kalohen?

Rasti 2b. Mismatch char nuk gjindet në pattern.

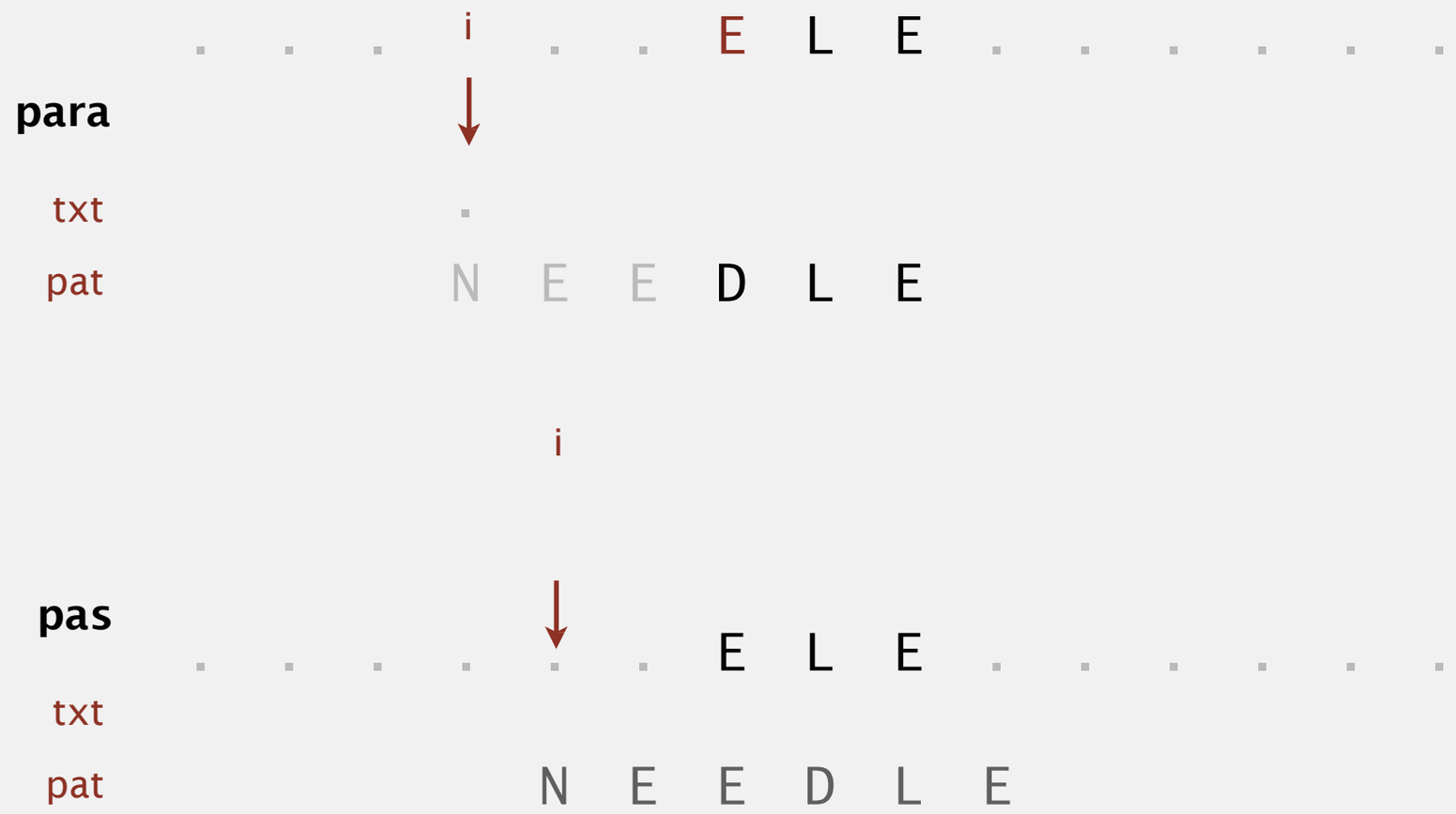


mismatch char 'E' në pattern: align 'E' e tekstit me 'E' më të djathtë në pattern?

Boyer-Moore

Pyetje. Sa karaktere të kalohen?

Case 2b. Mismatch character në pattern



mismatch character 'E' in pattern: Të rritet i për 1

Boyer-Moore

Pyetje. Sa karaktere të kalohen?

Përgjigje. Parallogaritet indeksi i paraqitjes së char c më të djathtë të mundshëm në pattern. (-1 nëse character nuk është në pattern)

```
right = new int[R];
for (int c = 0; c < R; c++)
   right[c] = -1;
for (int j = 0; j < M; j++)
   right[pat.charAt(j)] = j;
```

c		N	E	E	D	L	E
		0	1	2	3	4	5
A	-1	-1	-1	-1	-1	-1	-1
B	-1	-1	-1	-1	-1	-1	-1
C	-1	-1	-1	-1	-1	-1	-1
D	-1	-1	-1	-1	3	3	3
E	-1	-1	1	2	2	2	5
...							
L	-1	-1	-1	-1	-1	4	4
M	-1	-1	-1	-1	-1	-1	-1
N	-1	0	0	0	0	0	0
...							

Boyer-Moore skip table llogaritjet

Boyer-Moore Java implementation

```
public int search(String txt)
{
   int N = txt.length();
   int M = pat.length();
   int skip;
   for (int i = 0; i <= N-M; i += skip)
   {
      skip = 0;
      for (int j = M-1; j >= 0; j--)
      {
         if (pat.charAt(j) != txt.charAt(i+j))
         {
            skip = Math.max(1, j - right[txt.charAt(i+j)]);
            break;
         }
      }
      if (skip == 0) return i;
   }
   return N;
}
```

Ilogaritet vlera për kërcim

Nëse termi tjetër është jopozitiv

match



5.3 Substring Search

- *introduction*
- *brute force*
- *Knuth-Morris-Pratt*
- *Boyer-Moore*
- *Rabin-Karp*



Michael Rabin
Dick Karp

Rabin-Karp fingerprint search

Idea themelore = hashimi modular.

- Llogaritet një hash nga pat[0..M-1].
- Për çdo i, llogaritet një hash i txt[i..M+i-1].
- If pattern hash = text substring hash, kontrollohet për match.

```
          pat.charAt(i)
 i        0  1  2  3  4
-------------------------
          2  6  5  3  5   % 997 = 613
```

```
i        0  1  2  3  4   txt.charAt(i)
                         5  6  7  8  9 10 11 12 13 14 15
-------------------------------------------------------
         3  1  4  1  5   9  2  6  5  3  5  8  9  7  9  3
0        3  1  4  1  5   % 997 = 508
1           1  4  1  5   9  % 997 = 201
2              4  1  5   9  2  % 997 = 715
3                 1  5   9  2  6  % 997 = 971
4                    5   9  2  6  5  % 997 = 442
5                       9  2  6  5  3  % 997 = 929
6  <-- return i = 6           2  6  5  3  5  % 997 = 613
```

match

Hashimi modular, ku R = 10 dhe hash(s) = s (mod 997)

Shembull.

$$(10000 + 535) * 1000 \pmod{997}$$

$$= (30 + 535) * 3 \pmod{997}$$

$$= 1695 \pmod{997}$$

$$= 698 \pmod{997}$$

$$(a + b) \bmod Q = ((a \bmod Q) + (b \bmod Q)) \bmod Q$$

$$(a * b) \bmod Q = ((a \bmod Q) * (b \bmod Q)) \bmod Q$$

Identit modular aritmetik

Llogaritja e një hash function

Modular hash funkcioni. Përdoret notacioni t_i për `txt.charAt(i)`, që të llogaritet:

$$x_i \;=\; t_i\, R^{M-1} + t_{i+1}\, R^{M-2} + \ldots + t_{i+M-1}\, R^0 \pmod{Q}$$

Intuita. M-shifra, integer me bazë-R, modulo Q.

Metoda e Hornerit. Metodë në kohë lineare që të evaluohet polinomi I shkallës M.

	pat.charAt()				
i	0	1	2	3	4
	2	6	5	3	5

```
0   2   % 997 = 2
1   2   6   % 997 = (2*10 + 6) % 997 = 26
2   2   6   5   % 997 = (26*10 + 5) % 997 = 265
3   2   6   5   3   % 997 = (265*10 + 3) % 997 = 659
4   2   6   5   3   5   % 997 = (659*10 + 5) % 997 = 613
```

R Q

```
// Compute hash for M-digit key
private long hash(String key, int M)
{
   long h = 0;
   for (int j = 0; j < M; j++)
      h = (h * R + key.charAt(j)) % Q;
   return h;
}
```

26535 = 2*10000 + 6*1000 + 5*100 + 3*10 + 5

= ((((2) *10 + 6) * 10 + 5) * 10 + 3) * 10 + 5

Rabin-Karp substring search - shembull

R hyrjet e para: Përdoret metoda e Horner-it.

Hyrjet e tjera: Përdoret rolling hash (dhe % për të shmangur overflow).

```
 i    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
      3  1  4  1  5  9  2  6  5  3  5  8  9  7  9  3

 0    3  % 997 = 3                                  Q
 1    3  1  % 997 = (3*10 + 1) % 997 = 31
 2    3  1  4  % 997 = (31*10 + 4) % 997 = 314
 3    3  1  4  1  % 997 = (314*10 + 1) % 997 = 150
 4    3  1  4  1  5  % 997 = (150*10 + 5) % 997 = 508
 5       1  4  1  5  9  % 997 = ((508 + 3*(997 - 30))*10 + 9) % 997 = 201
 6          4  1  5  9  2  % 997 = ((201 + 1*(997 - 30))*10 + 2) % 997 = 715
 7             1  5  9  2  6  % 997 = ((715 + 4*(997 - 30))*10 + 6) % 997 = 971
 8                5  9  2  6  5  % 997 = ((971 + 1*(997 - 30))*10 + 5) % 997 = 442
 9                   9  2  6  5  3  % 997 = ((442 + 5*(997 - 30))*10 + 3) % 997 = 929
10  ← return i-M+1 = 6  2  6  5  3  5  % 997 = ((929 + 9*(997 - 30))*10 + 5) % 997 = 613
```

Metoda e Horner-it

RM R

rolling hash

match

-30 (mod 997) = 997 - 30

10000 (mod 997) = 30

Rabin-Karp: Java implementimi

```
public class RabinKarp
{
   private long patHash;    // pattern hash value
   private int M;           // pattern length
   private long Q;          // modulus
   private int R;           // radix   % Q
   private long RM1;        // R^(M-1)

   public RabinKarp(String pat) {
      M = pat.length();
      R = 256;
      Q = longRandomPrime();

      RM1 = 1;
      for (int i = 1; i <= M-1; i++)
         RM1 = (R * RM1) % Q;
      patHash = hash(pat, M);
   }

   private long hash(String key, int M)
   {  /* as before */  }

   public int search(String txt)
   {  /* see next slide */  }
}
```

një numër i thjeshtë (prime), i madh (but avoid overflow)

parallogaritet $R^{M-1} \pmod{Q}$

Rabin-Karp:Java implementimi (vazhdim)

Monte Carlo versioni. Return match nëse ka hash match.

```
public int search(String txt)
{
    int N = txt.length();
    int txtHash = hash(txt, M);
    if (patHash == txtHash) return 0;
    for (int i = M; i < N; i++)
    {
        txtHash = (txtHash + Q - RM*txt.charAt(i-M)  % Q) % Q;
        txtHash = (txtHash*R + txt.charAt(i)) % Q;
        if (patHash == txtHash) return i - M + 1;
    }
    return N;
}
```

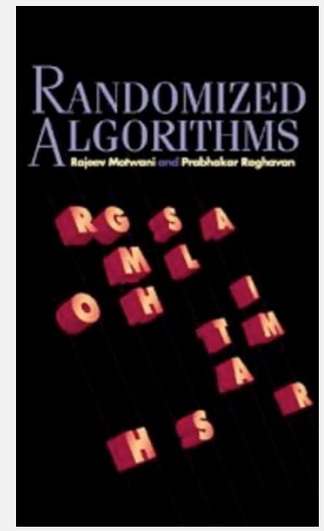
kontrollo për hash collision duke përdorur rolling hash funkcion

Las Vegas versioni. Kontrollo për substring match nëse ka hash match; vazhdo search nëse ka false collision.

Rabin-Karp analiza

Teoria. Nëse Q është një numër i theshtë i madh (rreth $M N^2$), atëherë probabiliteti I një false collision është rreth 1/N

Praktika. Zgjidhet Q që të jetë nr. i thjeshtë i madh (por jo I madh që të shkakton overflow). Nën këto kushte, probabiliteti i një collision është rreth $1 / Q$.



Monte Carlo versioni.

- Gjithmonë ekzekutohet në kohë lineare.
- Jashtëzakonisht e shpeshtë që jep përgjigjen e duhur

(mirëpo, jo gjithmonë!).

Las Vegas versioni.

- Gjithmonë jep përgjigjen e duhur.
- Jashtëzakonisht e shpeshtë që të ekzekutohet në kohë lineare

(mirëpo, është M N në rastin më të keq).

Rabin-Karp fingerprint search

Përparësitë.

- Zgjerohet për të gjetur 2D patterne.
- Zgjerohet për të gjetur patterne të shumëfishta.

Mangësitë.

- Operacionet aritmetike janë më të ngadalta sesa char krahasimet.
- Las Vegas versioni duhet të ketë backup.
- Në rastin më të keq garantohet shumë pak.



Substring search – përmbledhja e kostos

Cost of searching for an M-character pattern in an N-character text.

algorithm	version	operation count: guarantee	operation count: typical	backup in input?	correct?	extra space
brute force	—	MN	$1.1N$	*yes*	*yes*	1
Knuth-Morris-Pratt	*full DFA* (*Algorithm 5.6*)	$2N$	$1.1N$	*no*	*yes*	MR
	mismatch transitions only	$3N$	$1.1N$	*no*	*yes*	M
Boyer-Moore	*full algorithm*	$3N$	N/M	*yes*	*yes*	R
	mismatched char heuristic only (*Algorithm 5.7*)	MN	N/M	*yes*	*yes*	R
Rabin-Karp†	*Monte Carlo* (*Algorithm 5.8*)	$7N$	$7N$	*no*	*yes* †	1
	Las Vegas	$7N$ †	$7N$	*yes*	*yes*	1

† *probabilisitic guarantee, with uniform hash function*